

Οδηγός Προετοιμασίας

HY255 – Εργαστήριο Λογισμικού

Εξεταστική Ιουνίου 2026 – Διδάσκων: Α. Μπίλας

Με βάση ανάλυση παλιών θεμάτων (2023, 2024, 2025)

1. Γενική Μορφή της Εξέτασης

Καλά νέα

Ο Μπίλας είναι από τους πιο σταθερούς καθηγητές. Το pattern επαναλαμβάνεται σχεδόν αυτούσιο κάθε χρόνο. Αν δουλέψεις δομημένα τα 3 παλιά θέματα, θα έχεις δει ~90% του τι θα πέσει.

- **3 Ερωτήσεις (Α, Β, Γ)**, σύνολο ~180 βαθμοί.
- **Κλειστές σημειώσεις**.
- Διάρκεια: **3 ώρες** (09:30–12:30).
- Η εξέταση γίνεται **με χαρτί**, οπότε ψευδοκώδικας και διαγράμματα στοίβας πρέπει να είναι ευανάγνωστα.

Τυπική δομή ανά Ερώτηση:

- **Θέμα Α** (~50–60 βαθ.): βασικές γνώσεις C – τύποι, δηλώσεις, μνήμη, preprocessor.
- **Θέμα Β** (~60–70 βαθ.): πολυμορφικές συναρτήσεις, ADTs, FSM, λογικά λάθη.
- **Θέμα Γ** (~55–60 βαθ.): x86-32 memory layout, στοίβια, linking, assembly.

2. Συχνότητα Θεμάτων (2023-2025)

Θέμα	Εμφανίσεις	Σχόλιο
Memory layout σε σημείο L1	3/3	Πάντα 10 βαθμοί. ΕΓΓΥΗΜΕΝΟ.
Στοιβά σε σημείο L2 (x86-32)	3/3	14-21 βαθμοί. ΕΓΓΥΗΜΕΝΟ.
Symbols στο .o για linking	3/3	10 βαθμοί κάθε φορά.
Global δηλώσεις + μορφή στη μνήμη	3/3	10-30 βαθμοί. Σχεδόν εγγυημένο.
Δύο τρόποι για διευθύνσεις μεταβλητών	3/3	5-8 βαθμοί.
Πολυμορφική συνάρτηση (template-style)	2/3	25-30 βαθμοί.
Κλήση συνάρτησης σε x86-32 assembly	2/3	4-8 βαθμοί.
Preprocessor (header guards, #include)	2/3	10 βαθμοί.
setjmp / longjmp πρόγραμμα	2/3	15-35 βαθμοί όταν εμφανίζεται.
Μετατροπές τύπων σε εκφράσεις	2/3	10 βαθμοί.
Complex declarations (parse κανόνες)	2/3	function pointers, const, arrays.
Keywords της C (T/F)	2/3	5 βαθμοί - εύκολη ζέστανση.
Pointer arithmetic	2/3	5-15 βαθμοί.
FSM + υλοποίηση σε C	1/3	40 βαθμοί όταν εμφανίζεται (2025).
ADT design (bignum-style)	1/3	20 βαθμοί.
Λογικά λάθη σε snippets	2/3	15-30 βαθμοί.

3. Τα 6 «Βαρέλια» που Σχεδόν Σίγουρα θα Πέσουν

3.1 Memory Layout σε x86-32

Τι θα ζητηθεί

Δίνεται πρόγραμμα τύπου `layout.c` / `test.c`. Σου ζητάει την εικόνα της μνήμης σε σημείο L1 της εκτέλεσης. Πρέπει να σημειώσεις:

- Σε ποιο τμήμα της μνήμης βρίσκεται η κάθε μεταβλητή (**text**, **data**, **bss**, **heap**, **stack**).
- Τα περιεχόμενά της μετά την αρχικοποίηση.
- Συναρτήσεις και labels (L1, L2).

Πίνακας τμημάτων μνήμης (μάθε τον απ' έξω):

Τμήμα	Τι περιέχει	Παραδείγματα
text	κώδικας, read-only	συναρτήσεις, labels, string literals ("abc")
data	global/static αρχικοποιημένα $\neq 0$	static float x = 1.0;
bss	global/static μη αρχικοποιημένα ή = 0	double D[N];, unsigned int Sum;
heap	δυναμική μνήμη	malloc/calloc/realloc
stack	local variables, παράμετροι	ό,τι ζει μέσα σε function

3.2 Στοιβά σε σημείο L2 (x86-32)

Κλασικό λάθος

Στα x86-32, η στοίβα μεγαλώνει **προς τα κάτω** (από υψηλές προς χαμηλές διευθύνσεις). Όταν σχεδιάζεις, βάλε το `main` ψηλά και κάθε νέα κλήση **από κάτω**.

Τι μπαίνει στη στοίβα (cdecl convention, x86-32):

1. **Παράμετροι** της κληθείσας συνάρτησης (από δεξιά προς αριστερά).
2. **Return address** (από το `call`).
3. **Saved %ebp** του caller.
4. **Local variables** της συνάρτησης.

Όταν είναι αναδρομική (π.χ. `iter`, `f_init`, `ping/pong`), **ένα frame ανά κλήση**. Μετράς πόσες κλήσεις γίνονται μέχρι να φτάσει στο L2.

Παράδειγμα από JUNE25 (layout.c):

- `main` καλεί `iter(D, n-1) = iter(D, 2)`.
- `iter(D,2)` καλεί `iter(D,1)` καλεί `iter(D,0)` καλεί `iter(D,-1)`.
- Η `iter(D,-1)` κάνει `return` αμέσως.
- Στο L2 (`iter(D,0)` σταματάει εκεί), στη στοίβα: `main + 4 frames` της `iter`.

3.3 Symbols για Static Linking

Για κάθε σύμβολο, η πληροφορία που χρειάζεται ο linker:

- **Όνομα συμβόλου**
- **Τύπος:** defined (D) ή undefined (U) – δηλαδή αν ορίζεται εδώ ή χρειάζεται resolution.
- **Section:** text, data, bss.
- **Visibility:** global (extern, χωρίς static) ή local (static).
- **Offset / διεύθυνση** εντός του section.
- **Μέγεθος** (size).

Προσοχή στο static

static σε global → **δεν εμφανίζεται ως global symbol** στο .o (είναι local για τον linker). Δεν μπορεί να γίνει extern reference από άλλο αρχείο.

3.4 Δύο Τρόποι για Διευθύνσεις Μεταβλητών

Σχεδόν πανομοιότυπη ερώτηση κάθε χρόνο. Οι κλασικοί τρόποι:

1. **Στατικά:** τυπώνουμε με `printf("%p", &var);` ή χρησιμοποιούμε `nm a.out, objdump`.
2. **Δυναμικά:** μέσα στο πρόγραμμα, π.χ. `printf("%p", &var)`, debug με `gdb`.

Πάντα να απαντάς: «Ναι, αλλάζει η διεύθυνση από εκτέλεση σε εκτέλεση» – λόγω **ASLR** (Address Space Layout Randomization). **Εξάιρεση:** static/global μεταβλητές σε .data/.bss έχουν *σχετικές* σταθερές διευθύνσεις, αλλά η απόλυτη διεύθυνση αλλάζει λόγω randomization του base.

3.5 Πολυμορφική Συνάρτηση (Template-style)

To pattern

Συνάρτηση που δουλεύει με **οποιοδήποτε τύπο** χρησιμοποιώντας `void *` arrays και **callback function pointers** για συγκρίσεις/πράξεις.

Πρωτότυπο (όπως η minmax 2025 και init2d 2023):

```
void minmax(void *a, void *b, int n, size_t elem_size,
            int (*cmp)(const void *, const void *));
```

Βασικά σημεία στην υλοποίηση:

- Χρήση `void *` για generic pointers.
- Pointer arithmetic με byte offsets: `(char *)a + i * elem_size`.
- Callback function για σύγκριση/αντικατάσταση.
- Memory swap με `memcpy` ή byte-byte loop μέσω temp buffer.

Παράδειγμα κλήσης: για int arrays, ο χρήστης ορίζει `int int_cmp(const void *x, const void *y)` και την περνάει ως callback. Παρόμοιο pattern με `qsort` της `libc`.

3.6 FSM (Finite State Machine) σε C

To FSM της Άσκησης 1

Στο μάθημα έχετε κάνει FSM που διαβάζει input stream και αναγνωρίζει patterns (π.χ. comment removal). **Ξαναβλέπεις την Άσκηση 1** – ο Μπίλας ρητά αναφέρει «όπως κάναμε στην Άσκηση 1».

Βασική δομή υλοποίησης:

```
enum state { NORMAL, MAYBE_COMMENT, IN_COMMENT, ... };
enum state s = NORMAL;
int c;
```

```

while ((c = getchar()) != EOF) {
    switch (s) {
        case NORMAL:
            if (c == ':') s = MAYBE_COMMENT;
            else putchar(c);
            break;
        case MAYBE_COMMENT:
            if (c == ':') s = IN_COMMENT;
            else { putchar(':'); putchar(c); s = NORMAL; }
            break;
        case IN_COMMENT:
            if (c == '\n') { putchar('\n'); s = NORMAL; }
            break;
    }
}

```

Στο σχεδιάγραμμα του FSM βάλε: κόμβοι = states, ακμές = transitions με labels του χαρακτήρα εισόδου, και σημείωσε **output** σε κάθε ακμή (τι τυπώνεται).

4. Στρατηγική Προετοιμασίας

Σύνολο: ~45h για HY255 σε 4 εβδομάδες (το πιο «εύκολα προβλέψιμο» μάθημα)

Μπορείς να το ρίξεις και στις 35–40h αν αισθάνεσαι ότι κυλάει. Το ROI εδώ είναι το υψηλότερο από τα 4 μαθήματα.

4.1 Εβδομάδα 1 - Θεμέλια (12h)

- Πίνακας text/data/bss/heap/stack – κρυστάλλινος.
- Parse κανόνες για complex declarations (clockwise/spiral rule).
- Type conversion rules στη C (integer promotion, usual arithmetic conversions).
- Preprocessor: #include, #define, #ifdef/#ifndef, header guards.

4.2 Εβδομάδα 2 - Βάθος (12h)

- **x86-32 calling convention** (cdecl): παράμετροι σε στοίβα δεξιά → αριστερά, %eax για return value, %ebp/%esp.
- Σχεδιασμός στοίβας βήμα προς βήμα για αναδρομικές συναρτήσεις.
- Pointer arithmetic + void pointers + function pointers.
- setjmp/longjmp – πώς δουλεύει, πόσα stack frames δημιουργούνται.

4.3 Εβδομάδα 3 - Παλιά Θέματα υπό Συνθήκες (12h)

- Λύσε **και τα 3 παλιά θέματα** (2023, 2024, 2025) με χρονόμετρο, χωρίς σημειώσεις.
- Δώσε ιδιαίτερη προσοχή στο **Θέμα Γ** – αυτό σχεδόν δεν αλλάζει.

- Μετά από κάθε ένα: σύγκριση με τυχόν σημειώσεις/λύσεις, και σύνταξη cheat sheet με τα σημεία που χάθηκαν.

4.4 Εβδομάδα 4 - Εντατική (9h πριν την εξέταση)

- Ξανά-υλοποίηση μιας πολυμορφικής συνάρτησης από μνήμη.
- Ξανά-σχεδίαση ενός FSM από μνήμη.
- Ξανά-σχεδίαση ενός memory layout + stack από μνήμη.
- **ΟΧΙ νέο υλικό.**

5. Πρακτικές Συμβουλές Εξέτασης

1. **Ξεκίνα από το Θέμα Γ.** Είναι το πιο μηχανικό και το πιο προβλέψιμο - κερδίζεις γρήγορα 50+ βαθμούς και βάζεις χρόνο για τα δύσκολα.
2. **Σχεδιάζε καθαρά.** Στο memory layout και στη στοίβα, η ευκρίνεια του διαγράμματος μετράει όσο και η σωστή απάντηση. Χρησιμοποίησε χάρακα αν χρειάζεται.
3. **Πάντα γράφε τα τμήματα μνήμης:** σημείωσε δίπλα από κάθε μεταβλητή «data», «bss», «stack», «heap» - όχι μόνο τιμή.
4. **Στα complex declarations,** χρησιμοποίησε τον *clockwise/spiral rule*: ξεκίνα από το όνομα, κινήσου δεξιά ((), []), γύρισε αριστερά (*), συνέχισε με parentheses προς τα έξω. Παράδειγμα: `char * const *(*next)(void) = «next is a pointer to a function returning a pointer to a const pointer to char»`.
5. **Στην πολυμορφική συνάρτηση,** μην ξεχάσεις `size_t elem_size` - είναι το κλειδί για να δουλέψει με οποιοδήποτε τύπο.
6. **Διαχείριση χρόνου** σε 3 ώρες: ~45 λεπτά Α, ~60 λεπτά Β, ~60 λεπτά Γ, και 15 λεπτά review. Το Γ άξιζε λιγότερο από το Β στους τελευταίους χρόνους αλλά είναι πιο βαρύ σε χρόνο γραφής.

Παράρτημα Α: Quick-Reference Πινάκων

C Storage Classes & Linking

Δήλωση	Section	Linkage
<code>int x;</code> (global)	bss	external
<code>int x = 5;</code> (global)	data	external
<code>static int x;</code> (global)	bss	internal (file-local)
<code>static int x = 5;</code> (global)	data	internal (file-local)
<code>extern int x;</code>	— (declaration only)	resolves elsewhere
<code>int x;</code> (μέσα σε function)	stack	none (auto)
<code>static int x;</code> (μέσα σε function)	data/bss	none (persistent)
<code>const char *s = "abc";</code>	s: data/stack, "abc": text	—

Type Conversion Rules (Implicit)

1. **Integer promotion:** `char, short` → `int` σε εκφράσεις.

2. **Usual arithmetic conversions** (από κάτω προς τα πάνω):
`int` → `unsigned int` → `long` → `unsigned long` → `float` → `double`.
3. **Assignment**: ο τύπος της δεξιάς μετατρέπεται στον τύπο της αριστεράς.
4. **Pointer conversions** χρειάζονται `explicit cast` (εκτός `void *`).

x86-32 Calling Convention (cdecl)

- Παράμετροι: σε στοίβα, push **δεξιά** → **αριστερά**.
- Return value: στον καταχωρητή `%eax`.
- Caller cleans up the stack μετά το `call`.
- Callee-saved: `%ebx`, `%esi`, `%edi`, `%ebp`.
- Caller-saved: `%eax`, `%ecx`, `%edx`.

Σκελετός prologue/epilogue:

```

push    %ebp           # save old frame pointer
movl    %esp, %ebp     # new frame
subl    $N, %esp       # space for locals
...
movl    %ebp, %esp     # restore stack
pop     %ebp           # restore old %ebp
ret                     # pop return address into %eip

```

Spiral / Clockwise Rule για Declarations

```

char * const * (*next)(void);
               ^      ^
               |      +--- (), void: function taking void
               +--- *next: pointer to that function

```

Returning: `char * const * = pointer to const pointer to char`

Διαδικασία: ξεκίνα από το όνομα, κινήσου σπιδάλ προς τα έξω. Δεξιά πρώτα `()`, `[]`, αριστερά μετά `*`.